

Autonomous and Mobile Robotics

Lecture Notes

Tuo Nome

Anno Accademico 2025/2026

Preface

These notes were written during the *Autonomous and Mobile Robotics* course held by Prof. Giuseppe Oriolo at Sapienza University of Rome. The main purpose of this document is to provide a coherent and compact reference for the topics covered: kinematics of mobile robots, motion planning, motion control, localization, mapping, SLAM, and decision making under uncertainty. Theoretical derivations are interspersed with practical remarks, formulas, and, where useful, pseudocode or short implementations.

September 29, 2026

Contents

Preface	ii
1 Introduction: Applications, Problems, Architectures	1
1.1 Why Mobile Robots	1
1.2 Applications	1
1.3 The Key Problems of Mobile Robotics	1
1.4 Autonomy	2
1.5 Perception	2
1.6 Robot Architectures	2
1.6.1 Deliberative Architecture	2
1.6.2 Reactive Architecture	3
1.6.3 Hybrid Architectures	3
1.6.4 Behavior-Based Architecture	3
1.7 Course Approach	4
2 Configuration Space	5
2.1 Motivation: Why Configuration Space?	5
2.2 Modeling a Robot	5
2.2.1 Configuration and Degrees of Freedom	5
2.3 The Configuration Space and the Workspace	6
2.3.1 Workspace vs Configuration Space	7
2.4 Examples	7
2.4.1 Point Robot	7
2.4.2 Disk Robot	7
2.4.3 Robot with Orientation: $\mathbb{R}^2 \times \mathcal{S}^1$	7
2.4.4 Planar 2-Link Manipulator: $\mathcal{T}^2$	7
2.4.5 Rigid Body in 3D: $SE(3)$	7
2.5 Topology of Configuration Space	8
2.5.1 The 2R Planar Manipulator	8
2.5.2 Non-Injectivity of the Parameterization	9
2.5.3 Restoring Injectivity: the Fundamental Domain	9
2.5.4 The Remaining Problem: Closeness Is Not Preserved	9
2.5.5 Gluing the Square: Why C Is a Torus	10
2.5.6 Why Topology Matters	10
2.6 Constraints	11
2.7 C-Obstacles and Free Space	11
2.8 Relevance to Motion Planning	12
3 Cinematica	13
4 Pianificazione	14

5	Controllo	15
6	Localizzazione	16
7	Mapping	17
8	SLAM	18
9	Decisioni incertezza	19
10	Laboratorio	20
A	Formulario	21
B	Esercizi	22
C	Codice	23
C.1	Esempio: EKF SLAM	23
C.2	Esempio: RRT	23

List of Figures

1.1	Deliberative architecture.	3
1.2	Behavior-based architecture: elementary behaviors are coordinated by a fusion block (e.g. vector summation) before reaching the actuators. Adapted from [Cho+05].	4
2.1	The planar 2R arm and its (q_1, q_2) plane. Every point of the plane is a posture, but $(\bar{q}_1, \bar{q}_2)$ and $(\bar{q}_1 + 2\pi, \bar{q}_2)$ are different points describing the <i>same</i> posture: the parameterization is not injective.	8
2.2	Left: the half-open square $[0, 2\pi) \times [0, 2\pi)$ (dashed sides excluded) gives exactly one point per posture, so the representation is injective. Right: two points at opposite sides of the square are far apart in the plane but describe almost the same posture, so the square does not respect the topology of C	9
2.3	Gluing the opposite sides of the square: identifying $q_1 = 0 \equiv 2\pi$ (red) gives a cylinder, identifying $q_2 = 0 \equiv 2\pi$ (blue) closes it into a torus $\mathcal{T}^2 = \mathcal{S}^1 \times \mathcal{S}^1$. The points $q_1 = \varepsilon$ and $q_1 = 2\pi - \varepsilon$, far apart in the square, are now neighbours on either side of the seam: the torus respects the topology of C . Configuration spaces are in general manifolds (angular coordinates).	10

List of Tables

1.1	Comparison between fixed-base manipulators and wheeled mobile robots. The apparent simplicity of WMRs hides fundamental difficulties.	2
-----	---	---

Chapter 1

Introduction: Applications, Problems, Architectures

This introductory chapter frames the course. It does not introduce technical tools yet; its purpose is to define *what* mobile robotics is about, *why* it is hard, and *how* we will approach it.

1.1 Why Mobile Robots

Industrial fixed-base manipulators are fast and accurate, but they operate in a *limited, structured, known, static* workspace. To be useful outside the factory, robots must move in *large, unstructured, uncertain, dynamic* environments. This shift is what motivates the field of autonomous and mobile robotics.

The course focuses on *methods*, not on specific hardware or software: the goal is to master the basic planning and control techniques that enable mobility and autonomy on *any* mobile platform — wheeled, legged, flying, or underwater.

1.2 Applications

Mobile robots are already deployed in a wide spectrum of domains, which can be roughly divided into:

- **Structured environments (service robots):** cleaning, catering, customer assistance, surveillance, entertainment.
- **Unstructured environments (field robots):** planetary exploration, underwater monitoring, rescue, demining, agriculture, construction, military transportation.

The market for autonomous mobile robots is growing rapidly, with goods-to-person picking robots, self-driving forklifts, autonomous inventory robots, and unmanned aerial vehicles among the main segments.

1.3 The Key Problems of Mobile Robotics

Following Durrant-Whyte (1991), the field can be organized around three fundamental questions:

1. **Where am I?** — *localization*
2. **How am I supposed to get to the goal?** — *path / trajectory / motion planning*
3. **How do I actually move?** — *motion control*

The entire course is a declination of these three problems.

Why wheeled mobile robots are harder than they look. A useful comparison is between fixed-base manipulators and single-body wheeled mobile robots (WMRs):

	Fixed-base manipulator	WMR
1. Localization	easy (joint encoders)	difficult
2a. Path/trajectory planning	easy (all paths feasible)	difficult (not all paths feasible)
2b. Motion planning	difficult (many DOF)	more difficult (nonholonomic + DOF)
3. Motion control	difficult (inertial couplings)	more difficult (nonlinear, no smooth stabilizer)

Table 1.1: Comparison between fixed-base manipulators and wheeled mobile robots. The apparent simplicity of WMRs hides fundamental difficulties.

The real challenge, however, lies in *multi-body* mobile robots: mobile manipulators, tractor-trailer systems, and humanoids. These combine the planning and control difficulties of manipulators with the nonholonomic constraints and nonlinear dynamics of WMRs.

1.4 Autonomy

Definition 1.1 (Autonomy). *A robot is autonomous when it can solve the three key problems (localization, planning, control) in unstructured environments, under uncertain and possibly dynamic operating conditions.*

Progress in autonomy is easy to overestimate. The DARPA Grand Challenge (2005) demonstrated autonomous driving in the desert, and the DARPA Robotics Challenge (2015) demonstrated legged locomotion and manipulation in semi-structured scenarios. Yet real autonomy — especially when manipulation is involved — is still far from being solved.

1.5 Perception

Autonomy rests on *perception*, i.e. sensing plus interpretation. Two categories are distinguished:

- **Proprioceptive:** perception of the robot itself (position, orientation, velocity). Sensors: encoders, IMUs, GPS.
- **Exteroceptive:** perception of the environment (obstacles, other robots, people). Sensors: rangefinders, cameras, tactile sensors.

Perception is essential in unstructured environments, where the state of the robot and of the world cannot be assumed to be known.

1.6 Robot Architectures

Three paradigmatic ways of organizing perception, planning, and control on a robot:

1.6.1 Deliberative Architecture

“*Think, then act.*” The robot builds a model of the world, plans on it, and then executes. The classical pipeline is:

mission objectives $\rightarrow$ planning $\rightarrow$ path/trajectory $\rightarrow$ control

with perception feeding localization (SLAM) and planning, and control sending actuator commands to the physical robot.

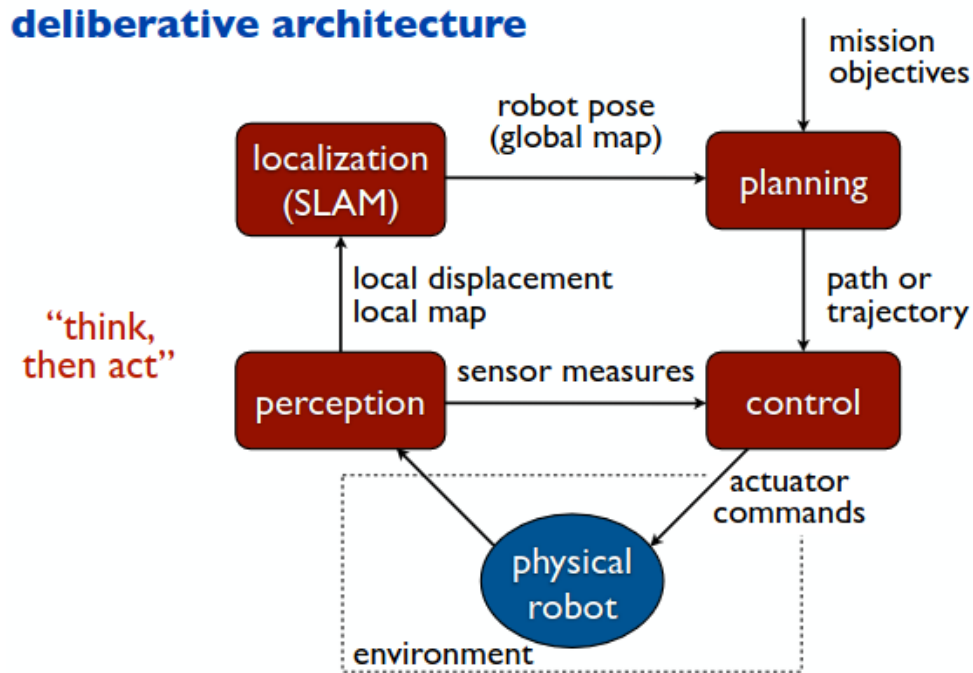


Figure 1.1: Deliberative architecture.

This is the reference architecture of the course.

1.6.2 Reactive Architecture

“Don’t think, (re)act.” No explicit model: direct mapping from sensor readings to actuator commands. Fast, but unable to handle non-trivial goals.

1.6.3 Hybrid Architectures

“Think and act concurrently.” A set of elementary behaviors (e.g. *avoid obstacles*, *follow wall*, *detect goal*) is coordinated by a fusion block (e.g. vector summation). Deliberative and reactive layers coexist, each handling what it does best. A deliberative layer handles long-horizon reasoning (mission planning, global localization), while a reactive layer handles short-horizon reflexes (obstacle avoidance, emergency stops). The two run in parallel and exchange information through a shared representation of the world.

1.6.4 Behavior-Based Architecture

“Think the way you act.” Rather than decomposing the problem into functional modules (perception, planning, control), a behavior-based architecture decomposes it into *behaviors*: elementary sensor-to-actuator mappings, each responsible for a specific task. Each behavior reacts to the current sensor readings and produces a *proposal* for the actuators. A **coordination/fusion** block combines these proposals into the final actuator command — for example, by vector summation, priority arbitration, or subsumption.

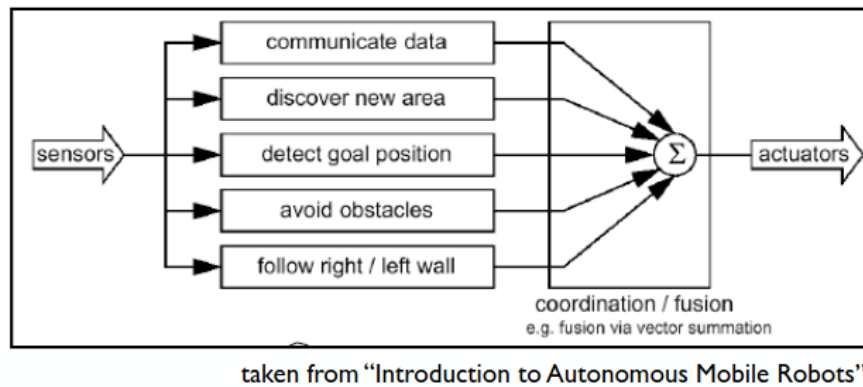


Figure 1.2: Behavior-based architecture: elementary behaviors are coordinated by a fusion block (e.g. vector summation) before reaching the actuators. Adapted from [Cho+05].

1.7 Course Approach

The course adopts a *model-based* approach, for the following reasons:

- Robots cannot escape physics.
- Models encode structured prior knowledge about the robot and the world — knowledge that does not need to be learned again.
- Machine learning can capture effects that are unknown a priori, difficult to model, or accessible only through data.
- Roboticists must be able to reason rigorously about autonomous physical systems, critically evaluate learned solutions, and combine the two.

Use models for what we know, data for what we don't know, and feedback for what is still missing.

Chapter 2

Configuration Space

The configuration space is the unifying abstraction of robot motion: once it is in place, planning, control, and localization all become statements about the same underlying object. This chapter introduces the concept, its structure, and its role in motion planning.

2.1 Motivation: Why Configuration Space?

The central problem of mobile robotics is the following: given a robot in an environment with obstacles, move it from an initial configuration to a final one without collisions. Stated directly in the workspace, this problem is hard: the robot has an extent, the obstacles have arbitrary shapes, and checking whether a given placement is collision-free requires geometric reasoning.

The *configuration space* (C-space) makes the problem tractable. The idea is to replace the motion of an extended body among obstacles with the motion of a *point* in an abstract space, where obstacles have been “inflated” to account for the robot’s shape. A robot is a continuous body, i.e. an infinite set of points. Describing it point by point is impossible. The key idea is to represent it with a *finite* set of parameters: the configuration. The solution is to define a suitable space (the C-space) in which *the robot is represented as a point*, and the obstacles are represented as regions of that space that the point cannot enter. This reduction turns a complex geometric problem into a path-search problem in a topological space, for which a large body of algorithms exists [Cho+05; Sic+25].

2.2 Modeling a Robot

A robot is a continuous body: an infinite set of points. Describing it point by point is impossible. The key idea is to find a *finite* set of parameters that completely determines the position of every point of the robot. We model a robot as a collection of n_b rigid bodies moving in a workspace W . The workspace is the set of physical points in which the robot can move.

2.2.1 Configuration and Degrees of Freedom

Definition 2.1 (Configuration). A configuration q is a complete specification of the position of every point of the robot. It is represented as a vector $q = [q_1, \dots, q_n]^\top$, where n is the number of degrees of freedom (DOF) of the robot.

The configuration vector $q = (q_1, \dots, q_n)$, where each q_i is a generalized coordinate (either Cartesian or angular). The Cartesian coordinates identify the position of points of the robot (e.g. the center of mass of a rigid body), and take values in $\mathbb{R}$. The angular coordinates identify the orientation of the rigid bodies, and take values in $SO(2)$ or $SO(3)$.

Thus, a *configuration* is a minimal set of parameters that identifies the posture of the robot in W : if the parameters are known, the position of every point of the robot is known.

Special Orthogonal Group $SO(n)$

The Special orthogonal group $SO(n)$ refers to the group of rotations in three-dimensional space that preserve the orientation and the Euclidean structure. It is characterized by orthogonal matrices with determinant equal to one, representing transformations that maintain distances and angles.

All the matrices R in $SO(n)$ satisfy the following properties:

- $R^\top R = I$, where $R^\top$ is the transpose of R and I is the identity matrix;
- $\det(R) = 1$, ensuring that the transformation preserves orientation;
- The set of all such matrices forms a group under matrix multiplication.

2.3 The Configuration Space and the Workspace

Definition 2.2 (Configuration Space). *The configuration space C is the set of all possible configurations of the robot. Formally, $C \subseteq \mathbb{R}^n$, although in many cases it carries a richer structure: for instance, $C = \mathbb{R}^2 \times \mathcal{S}^1$ for a planar robot with orientation, where $\mathcal{S}^1$ denotes the unit circle.*

For a collection of n_b rigid bodies moving in the plane, $\dim(C) = 3n_b$ (two translations and one rotation per body). In three dimensions, $\dim(C) = 6n_b$ (three translations and three rotations). For a manipulator with n joints, $\dim(C) = n$. In all cases,

$$\dim(C) = \text{DOF of the robot},$$

which in general differs from $\dim(W)$.

When such a structure of C is represented ($C = \mathbb{R}^i \times \mathcal{S}^j$), we are talking about the *Special Euclidean Group*.

Special Euclidean Group $SE(n)$

The Special Euclidean group $SE(n)$ describes the orientation-preserving isometries of the Euclidean space $\mathbb{R}^n$, where $n \in \mathbb{N}$ is an integer. It represents rigid transformations (combinations of rotations and translations) that preserve both distances between points and the orientation of the space.

This group can be represented using homogeneous transformation matrices as:

$$SE(n) = \left\{ X = \begin{pmatrix} R & p \\ 0_{1 \times n} & 1 \end{pmatrix} \in \mathbb{R}^{(n+1) \times (n+1)} \mid R^\top R = I_n, \det(R) = 1, p \in \mathbb{R}^n \right\}$$

The matrix components are defined as follows:

- **Rotation matrix (R):** An element of the Special Orthogonal group $SO(n)$, characterized by $R^\top R = I_n$ and $\det(R) = 1$, which handles changes in orientation without deformation or reflection.
- **Translation vector (p):** A vector $p \in \mathbb{R}^n$ representing the rigid displacement in space.
- **Bottom row ($0_{1 \times n}$ and 1):** A padding row that establishes homogeneous coordinates, allowing the composition of multiple rigid transformations through simple matrix multiplication.

By definition, the dimension of the configuration space C always equals the robot's degrees of freedom ($\dim(C) = \text{DOF}$), representing the minimum number of independent parameters

required to uniquely specify its state. *C-space* is the topological variety of all the possible configurations the robot can assume. In general, this does not coincide with the dimension of the workspace W ($\dim(W) \neq \dim(C)$), which depends on the physical task space (e.g., position in $\mathbb{R}^3$ or full pose in $SE(3)$) rather than the robot's kinematic structure.

2.3.1 Workspace vs Configuration Space

To recap:

- The **workspace** (W) is the physical space in which the robot moves (e.g. $\mathbb{R}^2$ or $\mathbb{R}^3$), expressed in world coordinates.
- The **configuration space** C is the abstract space of all configurations the robot can assume.

A planar robot that can both translate and rotate has a 3-dimensional C-space, even though it lives in a 2-dimensional workspace. A free rigid body in $\mathbb{R}^3$ has a 6-dimensional C-space, while its workspace is 3-dimensional.

2.4 Examples

2.4.1 Point Robot

If the robot is a point translating in $\mathbb{R}^2$, then $q = (x, y)$ and $C = \mathbb{R}^2$. The C-space coincides with the workspace, and the C-obstacles are the original obstacles.

2.4.2 Disk Robot

A disk-shaped robot of radius r translating in $\mathbb{R}^2$ has $q = (x, y)$, where (x, y) is the center of the disk. Again $C = \mathbb{R}^2$, but each obstacle is *inflated* by the robot's radius: a point obstacle becomes a disk of radius r , and a polygonal obstacle is expanded by the *Minkowski sum* with the robot's disk. After this inflation, the disk robot can be treated as a point in C .

2.4.3 Robot with Orientation: $\mathbb{R}^2 \times \mathcal{S}^1$

If the robot is a disk that can also rotate, then $q = (x, y, \theta)$ and $C = \mathbb{R}^2 \times \mathcal{S}^1$, where $\mathcal{S}^1$ is the unit circle (the angle θ is periodic). This space has a non-trivial topology: it is *not* $\mathbb{R}^3$, because θ and $\theta + 2\pi$ denote the same configuration. The intuitive picture is a *cylinder*: one linear dimension (x, y) and one circular dimension (θ) .

2.4.4 Planar 2-Link Manipulator: $\mathcal{T}^2$

For a planar manipulator with two revolute joints, $q = (\theta_1, \theta_2)$ and $C = \mathcal{S}^1 \times \mathcal{S}^1 = \mathcal{T}^2$, the *2-dimensional torus*. Each joint is a circle, and the product of two circles is a torus. This is a classic example of how the topology of C can differ radically from that of $\mathbb{R}^n$.

2.4.5 Rigid Body in 3D: $SE(3)$

A free rigid body in $\mathbb{R}^3$ has 6 DOF: three translational and three rotational. Its C-space is $C = \mathbb{R}^3 \times SO(3) = SE(3)$, the *special Euclidean group* of dimension 6. The rotational part $SO(3)$ is not $\mathbb{R}^3$: it is a compact 3-dimensional manifold.

2.5 Topology of Configuration Space

The configuration space C of a robot is not always a Euclidean space. Its *topology* — the way it is glued together globally — can differ substantially from that of $\mathbb{R}^n$, and this has direct consequences on the design of planning and control algorithms. The canonical example in this course is the planar 2R manipulator, which we use to answer three questions in order: what is C for this robot, why does the naive picture of it as a plane fail, and what is the right space to work in.

2.5.1 The 2R Planar Manipulator

Consider a planar manipulator with two revolute joints. Its configuration is $q = (q_1, q_2)^T$, where q_1 is the angle of the first link with respect to the base frame and q_2 the angle of the second link with respect to the first (fig. 2.1, left). Each joint angle is a planar rotation, i.e. an element of $SO(2) \cong \mathcal{S}^1$, the circle. The configuration space is therefore the product of two circles,

$$C = SO(2) \times SO(2) \cong \mathcal{S}^1 \times \mathcal{S}^1 = \mathcal{T}^2,$$

the *2-dimensional torus*.

The most immediate way to draw C is to plot every configuration as a point of the plane $(q_1, q_2) \in \mathbb{R}^2$ (fig. 2.1, right): each point is a posture of the arm. This picture is, however, misleading.

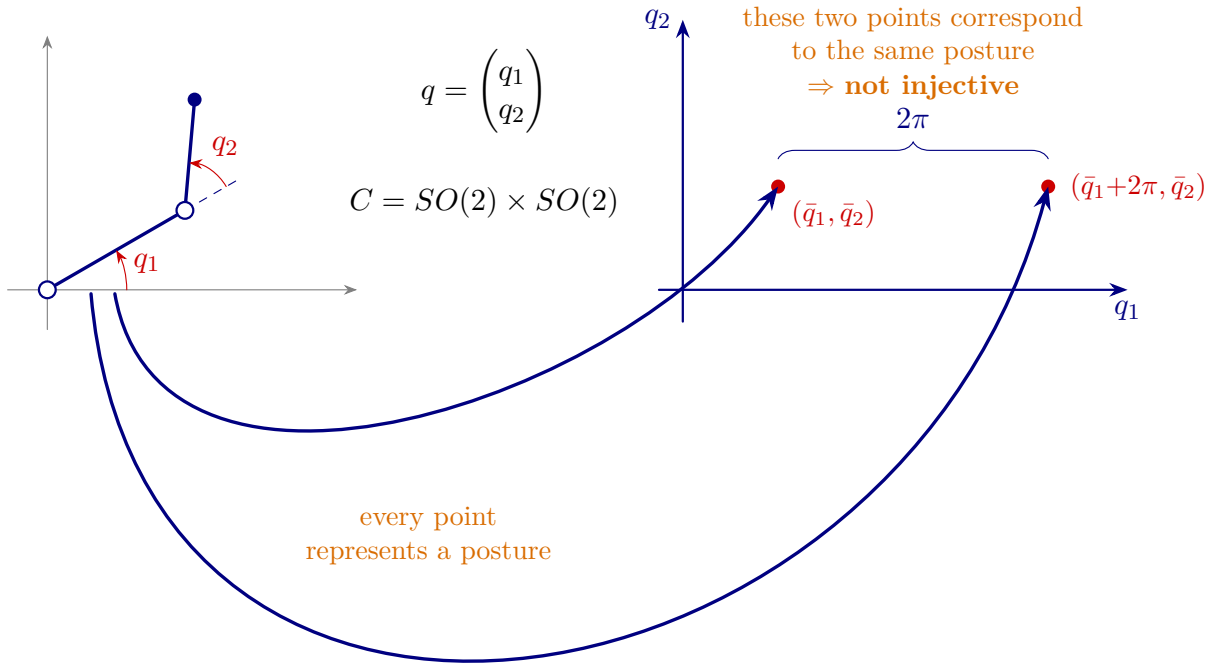


Figure 2.1: The planar 2R arm and its (q_1, q_2) plane. Every point of the plane is a posture, but $(\bar{q}_1, \bar{q}_2)$ and $(\bar{q}_1 + 2\pi, \bar{q}_2)$ are different points describing the *same* posture: the parameterization is not injective.

2.5.2 Non-Injectivity of the Parameterization

Angles are periodic with period 2π : rotating a joint by a full turn leaves the arm exactly where it was. The points $(\bar{q}_1, \bar{q}_2)$ and $(\bar{q}_1 + 2\pi, \bar{q}_2)$ of fig. 2.1 are therefore *different points describing the same posture*. Formally, the map

$$\Phi : \mathbb{R}^2 \rightarrow \mathcal{T}, \quad \Phi(q_1, q_2) = (q_1 \bmod 2\pi, q_2 \bmod 2\pi),$$

is *surjective but not injective*: two points have the same image if and only if they differ by integer multiples of 2π in each coordinate,

$$(q_1, q_2) \sim (q_1 + 2\pi k_1, q_2 + 2\pi k_2), \quad k_1, k_2 \in \mathbb{Z},$$

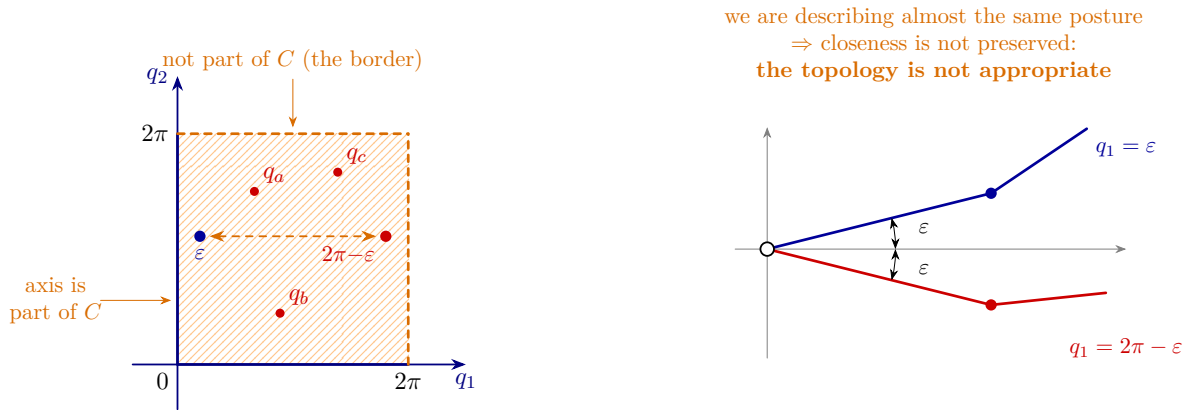
so every posture has infinitely many representatives in the plane. Restricting to the closed square $[0, 2\pi] \times [0, 2\pi]$ is not enough: opposite edges are still pairwise identified, $(q_1, 0) \sim (q_1, 2\pi)$ and $(0, q_2) \sim (2\pi, q_2)$, and the four corners all represent the same posture.

2.5.3 Restoring Injectivity: the Fundamental Domain

To remove the redundancy it is enough to keep one representative per class. The standard choice is the half-open square

$$[0, 2\pi) \times [0, 2\pi) \subset \mathbb{R}^2,$$

called the *fundamental domain*: the left and bottom edges belong to C , while the top and right edges do not (dashed in fig. 2.2, left). With this choice Φ becomes a *bijection*: for each configuration in the square there is one and only one posture, and every posture appears exactly once. This is why the fundamental domain is usually written as a half-open square.



for each configuration in this square there is only one posture
 $\Rightarrow$ new representation of C : **injective**

Figure 2.2: Left: the half-open square $[0, 2\pi) \times [0, 2\pi)$ (dashed sides excluded) gives exactly one point per posture, so the representation is injective. Right: two points at opposite sides of the square are far apart in the plane but describe almost the same posture, so the square does not respect the topology of C .

2.5.4 The Remaining Problem: Closeness Is Not Preserved

Injectivity is not enough. A good representation of C must also respect its *topology*: postures that are close must be represented by points that are close, and vice versa. The square fails this test. Take a small $\varepsilon > 0$ and consider $q_\varepsilon = (\varepsilon, q_2)$ and $q_{2\pi-\varepsilon} = (2\pi - \varepsilon, q_2)$ (fig. 2.2). In the

square they are almost as far apart as possible, at distance $2\pi - 2\varepsilon$; but the two arms differ only by a rotation of 2ε of the first joint, and as $\varepsilon \rightarrow 0$ they coincide (fig. 2.2, right). Equivalently, a continuous motion of the arm across $q_1 = 0$ (or $q_2 = 0$) appears in the square as a jump from one edge to the opposite one.

In mathematical terms, Φ restricted to the half-open square is a continuous bijection onto $\mathcal{T}^2$ whose inverse is discontinuous along the edges, so it is not a homeomorphism: the fundamental domain describes the same set of postures, but with the wrong topology.

2.5.5 Gluing the Square: Why C Is a Torus

The fix is to put back what the fundamental domain removed, namely the identifications $q_i = 0 \equiv 2\pi$ on the boundary, so that points on opposite edges become neighbours again. This can be done in two steps (fig. 2.3). Gluing the vertical edges $q_1 = 0 \equiv 2\pi$ turns the square into a cylinder; gluing the two circular boundaries $q_2 = 0 \equiv 2\pi$ of the cylinder closes it into a torus. The points q_ε and $q_{2\pi-\varepsilon}$ now lie on the two sides of the seam and are indeed close to each other: the torus respects the topology of C .

That the result is always a torus is not a coincidence but a consequence of the product structure: each joint contributes a circle, and the product of two circles is $\mathcal{T}^2$ (for n revolute joints, $\mathcal{T}^n$). The torus is also a *manifold*: around every point, seam included, a small neighbourhood looks like a disc of $\mathbb{R}^2$. Angular coordinates (q_1, q_2) are thus perfectly good *locally*, and the only obstruction is to use them *globally* with a single chart, which is what the square attempts to do. This is typical: configuration spaces are usually manifolds described by angular coordinates.

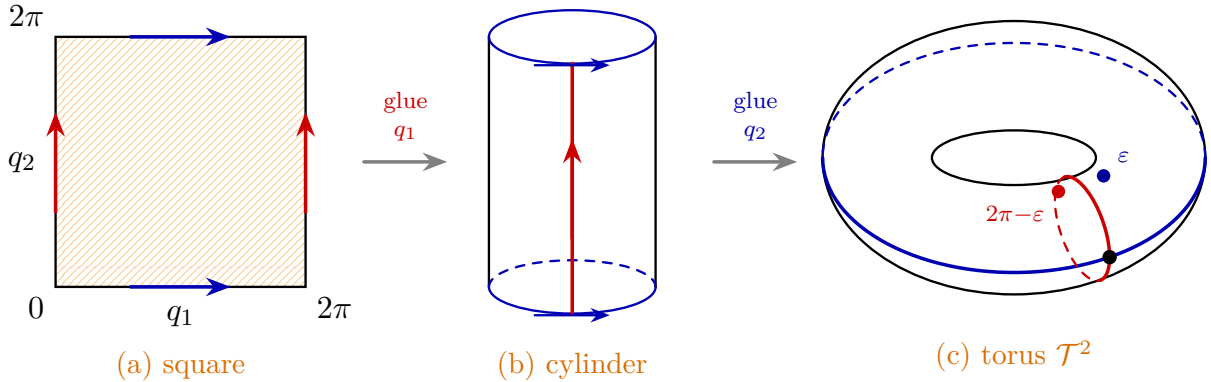


Figure 2.3: Gluing the opposite sides of the square: identifying $q_1 = 0 \equiv 2\pi$ (red) gives a cylinder, identifying $q_2 = 0 \equiv 2\pi$ (blue) closes it into a torus $\mathcal{T}^2 = \mathcal{S}^1 \times \mathcal{S}^1$. The points $q_1 = \varepsilon$ and $q_1 = 2\pi - \varepsilon$, far apart in the square, are now neighbours on either side of the seam: the torus respects the topology of C . Configuration spaces are in general manifolds (angular coordinates).

2.5.6 Why Topology Matters

The toroidal topology of C is not a mathematical curiosity: it has practical consequences.

- **Paths and interpolation.** A straight line in the square $[0, 2\pi]^2$ between q_ε and $q_{2\pi-\varepsilon}$ sweeps almost the whole range of the first joint, whereas on the torus the shortest path is the short arc across the seam, of length 2ε . Planners must wrap coordinates modulo 2π , not clamp them.
- **Distance metrics.** The Euclidean distance $\|q - q'\|$ is not the natural metric on $\mathcal{S}^1$: the correct angular distance is $\min(|q - q'|, 2\pi - |q - q'|)$.

- **Continuous parameterizations.** There is no globally continuous, single-valued parameterization of $\mathcal{S}^1$ by a real number: no interval of $\mathbb{R}$ is homeomorphic to the circle, so every real representation $q \in \mathbb{R}$ has a discontinuity at the wrap-around point. Controllers must account for this.

These issues generalize to any non-Euclidean configuration space: $\mathcal{T}^n$ for manipulators with revolute joints, $SE(3)$ for free-flying bodies, $\mathbb{R}^2 \times \mathcal{S}^1$ for wheeled mobile robots with orientation.

Torus vs. Sphere

The torus $\mathcal{T}^2$ and the sphere $\mathcal{S}^2$ are both 2-dimensional compact manifolds, but they are *not* homeomorphic: the torus has genus 1 (one hole), the sphere has genus 0. On the torus, a loop that wraps around the hole cannot be continuously contracted to a point; on the sphere, every loop can. This distinction is what makes the topology of C matter for planning.

2.6 Constraints

Not every configuration in C is reachable, and not every motion between two configurations is admissible: robots are subject to *constraints*. Two classes matter for this course.

Definition 2.3 (Holonomic Constraint). *A constraint is holonomic if it can be written as an algebraic relation among the configuration variables only:*

$$g(q) = 0.$$

Holonomic constraints reduce the dimension of the accessible configuration space: the robot is confined to a submanifold of C .

Definition 2.4 (Nonholonomic Constraint). *A constraint is nonholonomic if it involves the velocities and cannot be integrated into an algebraic relation among the configuration variables:*

$$a(q, \dot{q}) = 0, \quad \text{not equivalent to } g(q) = 0.$$

Nonholonomic constraints do not reduce the dimension of C , but they restrict the set of admissible velocities at each configuration. The robot can still reach any configuration, but not along arbitrary paths.

The typical example of a nonholonomic system is a car-like robot or a differential-drive robot: the no-sideways-slip condition on the wheels is a nonholonomic constraint. This is the fundamental reason why planning and control for wheeled mobile robots are harder than for fixed-base manipulators: the configuration space looks Euclidean, but the reachable motions are constrained.

2.7 C-Obstacles and Free Space

Once C is defined, obstacles from the workspace must be mapped into it.

Definition 2.5 (C-Obstacle). *Given an obstacle $O_i \subset W$, the C-obstacle CO_i is the set of configurations in which the robot collides with O_i :*

$$CO_i = \{q \in C : B(q) \cap O_i \neq \emptyset\},$$

where $B(q)$ is the volume occupied by the robot at configuration q .

The union of all C-obstacles is the *C-obstacle region*:

$$CO = \bigcup_i CO_i.$$

Definition 2.6 (Free Configuration Space). *The free configuration space is the complement of CO in C :*

$$C_{\text{free}} = C \setminus CO = \{q \in C : B(q) \cap \bigcup_i O_i = \emptyset\}.$$

Planning a safe motion is equivalent to finding a path in C_{free} connecting the start configuration q_s to the goal configuration q_g .

The point-robot trick. If the robot is modeled as a point, the workspace obstacles must be inflated by the Minkowski sum with the robot's geometry. After this transformation, planning for the extended robot reduces to planning for a point in C_{free} . This is the canonical formulation of the motion planning problem.

Path vs. trajectory. A *path* is a purely geometric sequence of configurations, with no timing information. A *trajectory* is a path endowed with a time law $q(t)$. In this chapter, and in motion planning in general, the object of interest is the path. Timing becomes relevant only when the dynamics and the control layer are brought into play.

2.8 Relevance to Motion Planning

The configuration space reformulates the motion planning problem in a clean and general way:

Given C , a start configuration $q_s \in C_{\text{free}}$, and a goal configuration $q_g \in C_{\text{free}}$, find a continuous path $\tau : [0, 1] \rightarrow C_{\text{free}}$ such that $\tau(0) = q_s$ and $\tau(1) = q_g$.

All the algorithms studied later in the course — roadmaps, cell decompositions, probabilistic planners, potential fields — are methods for solving this problem in C_{free} . The differences between them lie in how they represent C_{free} and how they search it.

A subtle point: the *topology* of C matters. Searching for a path in $\mathbb{R}^n$ is different from searching in $\mathcal{S}^1$, $\mathcal{T}^n$, or $SE(3)$. Planners must respect the topology of the space in which they operate.

Chapter 3

Cinematica

Chapter 4

Pianificazione

Chapter 5

Controllo

Chapter 6

Localizzazione

Chapter 7

Mapping

Chapter 8

SLAM

Chapter 9

Decisioni incertezza

Chapter 10

Laboratorio

Appendix A

Formulario

Appendix B

Esercizi

Appendix C

Codice

In questa appendice sono raccolti alcuni frammenti di codice utilizzati durante il corso.

C.1 Esempio: EKF SLAM

Listing C.1: Implementazione EKF SLAM

C.2 Esempio: RRT

Listing C.2: Implementazione RRT

Bibliography

- [Cho+05] Howie Choset et al. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.
- [Sic+25] Bruno Siciliano et al. *Foundations of Robotics*. Springer, 2025.